

Lumora Project Overview

1. Project Overview

Lumora is an AI-powered learning assistant provided as a Chrome Extension, designed specifically for YouTube videos. Its main objective is to help users learn, summarize, and interact with video content seamlessly. When a user watches a YouTube video, Lumora indexes its content and provides a suite of advanced learning tools directly within the browser.

Lumora uses a Retrieval-Augmented Generation (RAG) pipeline to analyze video transcripts. If a transcript is not available for a video, it gracefully falls back to native audio analysis, downloading the video's audio track and passing it directly to multimodal AI models.

2. Tech Stack

Frontend (Chrome Extension)

- **Framework:** React 19 (built with Vite)
- **Language:** TypeScript
- **Styling:** Tailwind CSS (v4) with Lucide React for iconography
- **Markdown Rendering:** react-markdown
- **Linting:** Oxlint
- **Build Tool:** Vite

Backend (REST API)

- **Framework:** FastAPI (Python)
- **Database (Relational):** PostgreSQL (or SQLite fallback) using SQLAlchemy ORM and Alembic for migrations, storing metadata, caching summaries, generated notes, quizzes, and chat history.
- **Database (Vector):** ChromaDB for semantic search and Retrieval-Augmented Generation (RAG) capabilities.
- **LLM Integration:** LangChain (langchain-huggingface, langchain-chroma, langchain-google-genai).
- **AI Models:**
 - **Embeddings:** HuggingFace BAAI/bge-base-en-v1.5 for creating semantic vector representations of transcript chunks.
 - **Generative AI:** Google Gemini (gemini-1.5-flash and gemini-flash-lite-latest depending on text vs multimodal audio tasks).
- **Audio Processing:** yt-dlp to fetch lowest quality audio streams for videos without captions.

3. Key Features

Lumora offers a comprehensive set of features separated into individual tabs within the extension:

- **Chat (RAG-powered Q&A):** Users can ask specific questions about the video. The backend retrieves the most relevant context using ChromaDB and answers using Gemini. It cites exact timestamps (e.g., 📍 [MM:SS]) so users can jump to the relevant part of the video.
- **Summary:** Generates executive, detailed, bulleted, ELI5 (Explain Like I'm 5), or rapid revision summaries.
- **Notes:** Generates structured notes with dedicated sections such as Introduction, Main Concepts, Definitions, Examples, Key Takeaways, and Interview Questions.
- **Quiz:** Generates a 20-question JSON-structured interactive quiz containing multiple-choice, true/false, and fill-in-the-blank questions.
- **Timeline:** Analyzes the full context to extract major topics and timestamps across the entire duration of the video.
- **Flashcards:** Extracts the most important concepts and creates a set of Question & Answer flashcards for quick revision.

4. Implementation Details

- **Video Processing Pipeline:**
 1. When a video is opened, the frontend fetches the video ID and sends it to the backend `process_video` endpoint.
 2. The backend tries to fetch the transcript using the `TranscriptService`.
 3. If successful, the transcript is chunked (~1000 characters with ~200 character overlaps preserving timestamps) and embedded into ChromaDB using HuggingFace embeddings.
 4. If the transcript fetch fails, the `AudioService` falls back to downloading the video's audio using `yt-dlp` and uploads the raw audio to Google Gemini via its File API for native multimodal analysis.
- **Context-Aware Chat:**
 - Implements a history-aware retriever using `LangChain RunnableBranch`. It reformulates user questions based on recent chat history to maintain conversation flow.
 - Generates responses in a streaming manner using `StreamingResponse` in FastAPI to ensure low latency and real-time feel.
- **Caching Mechanism:**
 - Results of computationally expensive tasks (like generating notes, flashcards, quizzes, and summaries) are cached in the SQLite database (`lumora.db`). This prevents redundant API calls to Gemini and allows users to revisit content instantly.
- **Responsive & Modern UI:**
 - The extension UI is constructed with a modern, glassmorphic aesthetic using Tailwind CSS.
 - Dynamic tab navigation allows fast switching between Chat, Notes, Summary, Quiz, Timeline, and Flashcards.

5. Production-Readiness Enhancements

Recent updates have transformed Lumora from a prototype to a production-ready system:

- **Asynchronous Processing:** Long-running tasks, particularly downloading missing audio tracks using `yt-dlp` and uploading them to the Gemini API, are now offloaded to FastAPI BackgroundTasks. The frontend intelligently polls a new status endpoint, presenting a seamless loading UI.
- **RAG Evaluation Framework:** A dedicated suite exists to test, score (1-5 scale via LLM-as-a-judge), and evaluate the Retrieval-Augmented Generation responses using mock question/answer pairs.
- **Robust Rate Limiting:** An in-memory sliding window rate limiter protects resource-intensive API endpoints, limiting requests based on a unique client install ID or IP address to prevent abuse.
- **Scalable Database:** Transitioned from a simple SQLite configuration to dynamic connection logic supporting PostgreSQL, fully equipped with Alembic for automated schema migrations.